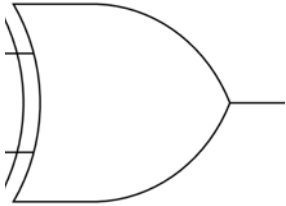


A 1.2.5 Logic Diagrams



1.2.5 Construct logic diagrams

A logic diagram is a drawing of a circuit built from logic gates, with wires carrying binary signals from the inputs on the left to the output on the right. Learning to read and draw these diagrams is how you move between three views of the same logic: the truth table, the Boolean expression, and the circuit. Each view states the same relationship in a different language, and a question can ask you to translate from any one to any other. This page sets out the seven standard gate symbols, shows how a signal is traced through a diagram to produce an output, builds the gates into a useful circuit in the half adder, and applies the rules of Boolean algebra to reduce a diagram to fewer gates.

Last Updated: 15 July 2026

Key Questions

- How does a logic diagram show the way gates are connected and interact in a circuit?
- What are the standard symbols for the AND, OR, NOT, NAND, NOR, XOR and XNOR gates?
- How do you trace a set of inputs through a diagram to find its output?
- How do gates combine to carry out more complex operations such as binary addition?
- How do the rules of Boolean algebra simplify a logic diagram or expression?

IB Command Term note

The command term in this learning statement is **construct**. In IB terms, construct means to display information in a diagrammatic or logical form, so a construct question expects a drawn diagram, not a description in words.

Marks come from using the correct symbol for each gate, connecting the gates so the signal path is unambiguous, and labelling every input and the output.

When you are given an expression, work from the inside of the brackets outwards; when you are given a truth table, identify the gate or combination that reproduces every row.

A neat diagram with the right symbols will get you the marks. A paragraph describing what you would draw does not.

How a logic diagram represents a circuit

A **logic diagram** represents a circuit as a set of gates joined by wires. Each wire carries a single binary value, 0 or 1, and signals flow from left to right.

Inputs enter on the left, pass through the gates, and emerge as one or more outputs on the right.

The **output** of one gate can become the input of another, which is how small gates build into larger circuits.

Two drawing **conventions** decide whether your diagram means what you intend. Inputs are labelled with letters, usually A, B and C, and the final output with a letter such as Z.

Where two wires meet and join, a solid dot marks the connection.

Where two wires cross without a dot it means they are not connected. A dot in the wrong place, or a missing one, changes the circuit, so draw junctions carefully.

The standard gate symbols

Seven gates need to be recognised and drawn. Three are the building blocks, AND, OR and NOT, and the other four are built from these by inverting an output or by testing whether the inputs differ.

THINK FIRST

You already know that an AND gate outputs 1 only when both inputs are 1. Before reading on, predict: if you invert that output, for which input pair will the result now be 0? That inverted gate is the NAND, and your prediction is its truth table.



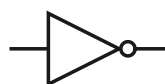
AND

1 only if both inputs are 1



OR

1 if either input is 1



NOT

inverts the single input



NAND

NOT AND: the inverse of AND



NOR

NOT OR: the inverse of OR



XOR

1 only if the inputs differ



XNOR

1 only if the inputs match

The seven standard logic gate symbols and what each one does.

The small circle, or bubble, on a gate's output means invert. Add it to AND and you have NAND; add it to OR and you have NOR; add it to XOR and you have XNOR. NOT is a bubble on its own, drawn on a triangle. This is the most useful pattern on the page: learn three gates and the rule that a bubble inverts, and you have six of the seven.

XOR and XNOR are the odd pair. XOR outputs 1 only when the two inputs differ, and its second curved line at the back is the only thing distinguishing it from OR. XNOR is the inverted XOR, so it outputs 1 only when the inputs match. The behaviour of all six two-input gates is set out below; NOT, with its single input, follows after.

A	B	AND	OR	NAND	NOR	XOR	XNOR
0	0	0	0	1	1	0	1
0	1	0	1	1	0	1	0
1	0	0	1	1	0	1	0
1	1	1	1	0	0	0	1

A	NOT A
0	1
1	0

COMMON EXAM MISTAKE

Drawing an OR symbol where the circuit needs an XOR, or the reverse. The two shapes differ only by the extra curved line at the back of the XOR, and it is easy to leave off when sketching at speed. The gates

disagree on the input pair 1, 1: OR gives 1, XOR gives 0. Check the back of every OR-shaped gate you draw.

Drawing a diagram from a Boolean expression

To construct a diagram from a Boolean expression, work from the inside of the brackets outwards, in the order the expression is evaluated.

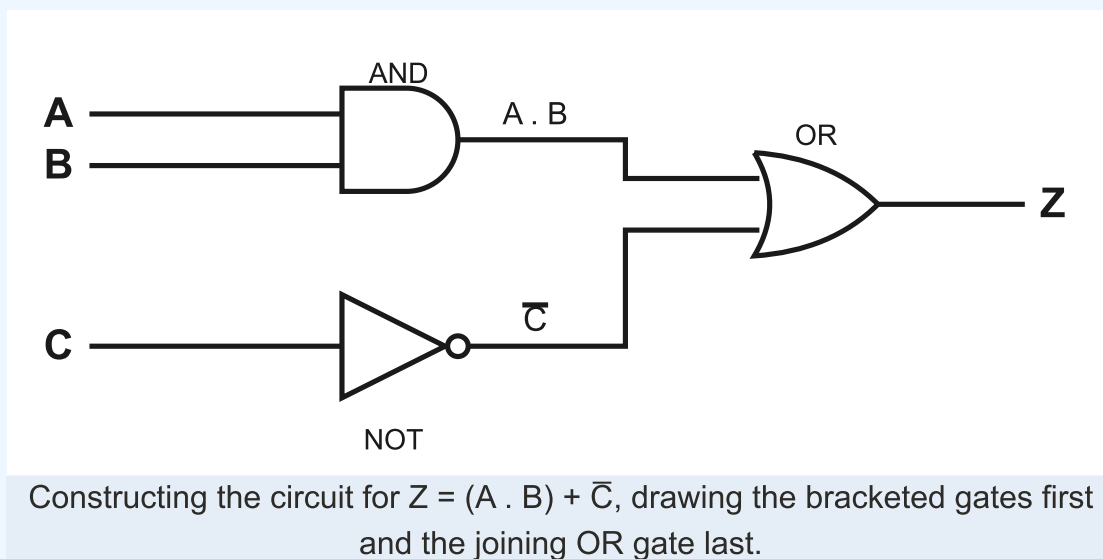
Take $Z = (A \cdot B) + \bar{C}$.

The bracket $(A \cdot B)$ is an AND of A and B, so it needs one AND gate.

The term $\bar{C}$ is C inverted, so it needs a NOT gate on input C.

The plus joins the two results, so an OR gate takes the AND output and the NOT output and produces Z.

Drawing the inner gates first and the joining gate last keeps the wiring straight.



Reading the finished diagram back confirms the construction: the AND output is labelled $A \cdot B$, the NOT output is $\bar{C}$, and the OR combines them into Z. If your diagram reads back as the original expression, it is correct.

EXAM TIP

Label the output of every intermediate gate with the sub-expression it produces, as in the diagram above. Your own checking is faster, and if the final wiring is wrong an examiner can still see that you formed the parts correctly. Unlabelled wires force the reader to re-derive your intention.

Tracing inputs through a diagram to find the output

Once a diagram is drawn, you find its output for any set of inputs by tracing values through it. Write each input value on its wire, then move left to right, applying each gate to the values on its input wires and writing the result on its output wire. The value on the final wire is the output. The method is mechanical, which is the point: there is no need to reason about the whole circuit at once, only to apply one gate at a time.

WORKED EXAMPLE

Tracing the circuit for $Z = (A \cdot B) + \bar{C}$.

Take $A = 1$, $B = 0$, $C = 1$.

- AND gate: $1 \cdot 0 = 0$, so the upper wire carries 0.
- NOT gate: $\bar{C} = \text{NOT } 1 = 0$, so the lower wire carries 0.
- OR gate: $0 + 0 = 0$, so $Z = 0$.

Now take $A = 1$, $B = 1$, $C = 0$.

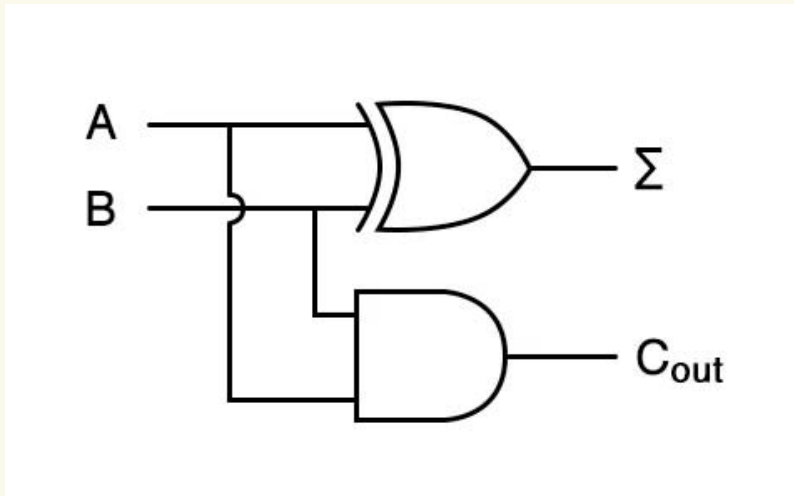
- AND gate: $1 \cdot 1 = 1$.
- NOT gate: $\text{NOT } 0 = 1$.
- OR gate: $1 + 1 = 1$, so $Z = 1$.

The same three steps work for any of the eight input combinations.

Combining gates: the half adder

Useful circuits come from combining gates so that the output of one feeds the input of another. The first such circuit worth knowing is the half adder, which adds two single bits.

It has two outputs, the Sum and the Carry, because adding 1 and 1 in binary gives 10: a sum digit of 0 and a carry of 1.



The half adder. The Sum output is $A \text{ XOR } B$ and the Carry output is $A \text{ AND } B$.

A	B	Sum	Carry
0	0	0	0
0	1	1	0
1	0	1	0
1	1	0	1

Read the Sum column and it is exactly the XOR truth table; read the Carry column and it is exactly the AND.

So the half adder is two gates sharing the same two inputs: an XOR for the sum and an AND for the carry.

It is called a *half* adder because it produces a carry out but cannot accept a carry in, so on its own it cannot add a column in the middle of a multi-bit number.

Simplifying with Boolean algebra

Boolean algebra is a set of rules for rewriting a logical expression as an equivalent one, usually a shorter one. A shorter expression means fewer gates, which in real hardware means less power drawn and less chip area used, so simplification is not merely tidy, it is cheaper to build.

The **notation** carries over from the diagrams: a dot for AND ($A \cdot B$), a plus for OR ($A + B$), and a bar over a variable for NOT ($\bar{A}$).

The **order of evaluation**, highest priority first, is NOT, then AND, then OR, and anything inside brackets is evaluated before anything outside.

The laws of Boolean algebra

Law	AND form	OR form
Annulment	$A \cdot 0 = 0$	$A + 1 = 1$
Identity	$A \cdot 1 = A$	$A + 0 = A$
Idempotent	$A \cdot A = A$	$A + A = A$
Complement	$A \cdot \bar{A} = 0$	$A + \bar{A} = 1$
Commutative	$A \cdot B = B \cdot A$	$A + B = B + A$
Associative	$A \cdot (B \cdot C) = (A \cdot B) \cdot C$	$A + (B + C) = (A + B) + C$
Distributive	$A \cdot (B + C) = (A \cdot B) + (A \cdot C)$	$A + (B \cdot C) = (A + B) \cdot (A + C)$
Absorption	$A \cdot (A + B) = A$	$A + (A \cdot B) = A$

Two further rules come up constantly. De Morgan's theorem pushes a NOT through a bracket: $\text{NOT}(A \cdot B) = \bar{A} + \bar{B}$, and $\text{NOT}(A + B) = \bar{A} \cdot \bar{B}$. The operation flips between AND and OR, and each variable is inverted.

Double negation cancels: $\text{NOT}(\text{NOT } A) = A$.

One pattern is worth spotting on sight, since it appears in the first worked example below: $A \cdot B + A \cdot \bar{B} = A$, because B and $\bar{B}$ together cover both cases and so drop out.

WORKED EXAMPLE

Simplify $(A \cdot B) + (A \cdot \bar{B}) + C$.

- Start: $(A \cdot B) + (A \cdot \bar{B}) + C$
- Factor A from the first two terms (distributive law): $A \cdot (B + \bar{B}) + C$
- $B + \bar{B} = 1$ (complement law): $A \cdot 1 + C$
- $A \cdot 1 = A$ (identity law): $A + C$

Simplified result: $A + C$. The circuit reduces to a single OR gate on A and C.

WORKED EXAMPLE

Simplify $(A + B) \cdot (A \cdot \bar{C})$.

- Start: $(A + B) \cdot (A \cdot \bar{C})$
- Expand (distributive law): $(A \cdot A \cdot \bar{C}) + (B \cdot A \cdot \bar{C})$
- $A \cdot A = A$ (idempotent law): $(A \cdot \bar{C}) + (A \cdot B \cdot \bar{C})$
- Factor $A \cdot \bar{C}$: $A \cdot \bar{C} \cdot (1 + B)$
- $1 + B = 1$ (annulment law): $A \cdot \bar{C} \cdot 1$
- $A \cdot \bar{C} \cdot 1 = A \cdot \bar{C}$ (identity law)

Simplified result: $A \cdot \bar{C}$. There is often more than one valid route to the simplest form; any sequence of correct law applications that reaches it is acceptable.

EXAM TIP. You are **not** expected to memorise the laws of Boolean algebra. Any law a question needs, such as De Morgan's, is given to you in the question. What is marked is choosing the right law, applying it correctly, and extending it where the expression is larger, for example from two variables to three. Learn to use the laws and to spot which one fits, rather than to recite them.

SUMMARY

Gates and behaviour:

- AND: 1 only when both inputs are 1 ($A \cdot B$)
- OR: 1 when either input is 1 ($A + B$)
- NOT: inverts the single input ($\bar{A}$)
- NAND, NOR, XNOR: AND, OR and XOR with the output inverted (an output bubble)
- XOR: 1 only when inputs differ; XNOR: 1 only when inputs match

Reading diagrams: signals flow left to right; a solid dot joins wires, a plain crossing does not.

Tracing: write each input value on its wire and apply one gate at a time, left to right.

Half adder: Sum = $A \text{ XOR } B$, Carry = $A \text{ AND } B$.

Boolean order of evaluation: brackets, then NOT, then AND, then OR.

Key laws: identity ($A \cdot 1 = A$), complement ($A + \bar{A} = 1$), De Morgan ($\text{NOT}(A \cdot B) = \bar{A} + \bar{B}$).

TRY IT ON PAPER

Answer these before the quiz, then check the model answers.

1. Construct a logic diagram for $Z = (A \cdot B) + \bar{C}$. **[3 marks]**

Model answer: an AND gate with inputs A and B [1]; a NOT gate with input C [1]; an OR gate taking the AND output and the NOT output to give Z [1].

2. Use De Morgan's theorem to rewrite $\text{NOT}(A + B)$. **[2 marks]**

Model answer: $\text{NOT}(A + B) = \bar{A} \cdot \bar{B}$ [1]; the OR becomes an AND and each variable is inverted [1].

3. Simplify $(A \cdot B) + (A \cdot \bar{B}) + C$, naming the law used at each step. **[4 marks]**

Model answer: factor A from the first two terms to get $A \cdot (B + \bar{B}) + C$ [1]; apply the complement law, $B + \bar{B} = 1$, giving $A \cdot 1 + C$ [1]; apply the identity law, $A \cdot 1 = A$, giving $A + C$ [1]; state the simplified result $A + C$ [1].

4. State the Boolean expressions for the Sum and Carry outputs of a half adder. **[2 marks]**

Model answer: Sum = $A \text{ XOR } B$ [1]; Carry = $A \cdot B$, that is A AND B [1].

A 1.2.5 Quiz

1

Which gate outputs 1 only when both of its inputs are 1?

- A. ☐ OR
- B. ☒ AND
- C. ☐ NAND
- D. ☐ XOR

AND outputs 1 only for the input pair 1, 1. OR outputs 1 whenever at least one input is 1, so it is the trap for anyone who reads "both" as "either".

2

A NAND gate receives inputs $A = 1$ and $B = 1$. What is its output?

A. ☒ 0

B. ☐ 1

C. ☐ It depends on a third input

D. ☐ Undefined

NAND is AND followed by inversion. AND of 1 and 1 is 1, and inverting gives 0.

3

An XOR gate outputs 1 when:

A. ☐ both inputs are 1

B. ☐ both inputs are 0

C. ☒ the two inputs are different

D. ☐ the two inputs are the same

XOR outputs 1 only when the inputs differ. Option D describes XNOR, the inverted XOR, which is the answer chosen by anyone who has swapped the two gates.

4

In the expression $A + B \cdot C$, which operation is evaluated first?

A. ☐ $A + B$

B. ☒ $B \cdot C$

C. ☐ Left to right, so $A + B$

D. ☐ It makes no difference

AND has higher priority than OR, so $B \cdot C$ is evaluated before the OR. Reading strictly left to right, option C, is the common error and produces the wrong circuit.

5

Which law states that $A + \bar{A} = 1$?

A. ☐ Identity law

B. ☐ Idempotent law

C. ☒ Complement law

D. ☐ Annulment law

The complement law: a variable ORed with its own inverse is always 1, because one of the two must be 1. Identity ($A + 0 = A$) is the distractor for anyone who confuses which constant is involved.

6

NOT($A \cdot B$) is equivalent to:

A. ☐ $\bar{A} \cdot \bar{B}$

B. ☒ $\bar{A} + \bar{B}$

C. ☐ $A \cdot B$

D. ☐ $A + B$

De Morgan's theorem: the NOT of an AND is the OR of the NOTs. Option A inverts the variables but keeps the operation as AND, which is the usual slip.

7

In a logic diagram, two wires cross but there is no solid dot at the crossing. This means:

- A. ☐ the wires are connected
- B. ☒ the wires are not connected
- C. ☐ the diagram is drawn incorrectly
- D. ☐ one wire carries 0

A connection is shown only by a solid junction dot; a plain crossing carries two independent signals. This convention is what lets one diagram route several signals past each other without joining them.

8

The Carry output of a half adder is produced by which gate?

- A. ☐ XOR
- B. ☐ OR
- C. ☒ AND
- D. ☐ NOT

Carry is 1 only when both inputs are 1, which is the AND function. The Sum, not the Carry, uses XOR, so option A is the natural distractor.

9

Simplify $A \cdot (A + B)$

- A. ☒ A
- B. ☐ B

C. ☐ $A \cdot B$

D. ☐ $A + B$

The absorption law: $A \cdot (A + B) = A$. If A is 0 the whole expression is 0, and if A is 1 the result follows A whatever B is, so the expression depends on A alone.

10

Which single change turns an OR symbol into a NOR symbol?

A. ☐ Adding a second curved line at the back

B. ☒ Adding a small circle (bubble) at the output

C. ☐ Removing one input

D. ☐ Making the front pointed

A bubble on the output inverts it, turning OR into NOR. Option A describes the change that makes an XOR, the distractor for anyone who confuses the two modifications.

All materials on this website are for the exclusive use of teachers and students at subscribing schools for the period of their subscription. Any unauthorised copying or posting of materials on other websites is an infringement of our copyright and could result in your account being blocked and legal action being taken against you.

 **Report a problem**